

Advanced SQL

Recap

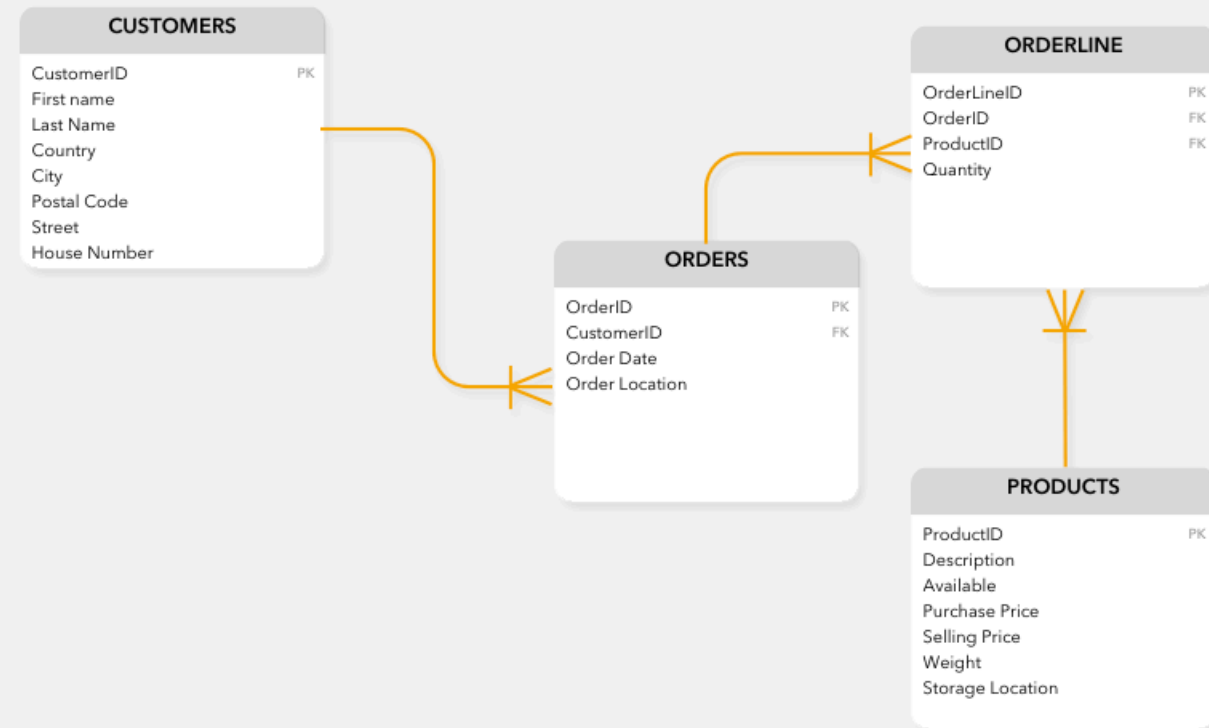
- `SELECT` columns `FROM` a table `WHERE` conditions are true
 - `LIKE` pattern matching operator (with `%` and `_`)
 - `IN` membership operator
 - `BETWEEN` range operator
- `LIMIT` the number of records returned
- `ORDER BY` columns

Sample ERD

Entities: Customers, Orders, Products, Orderline

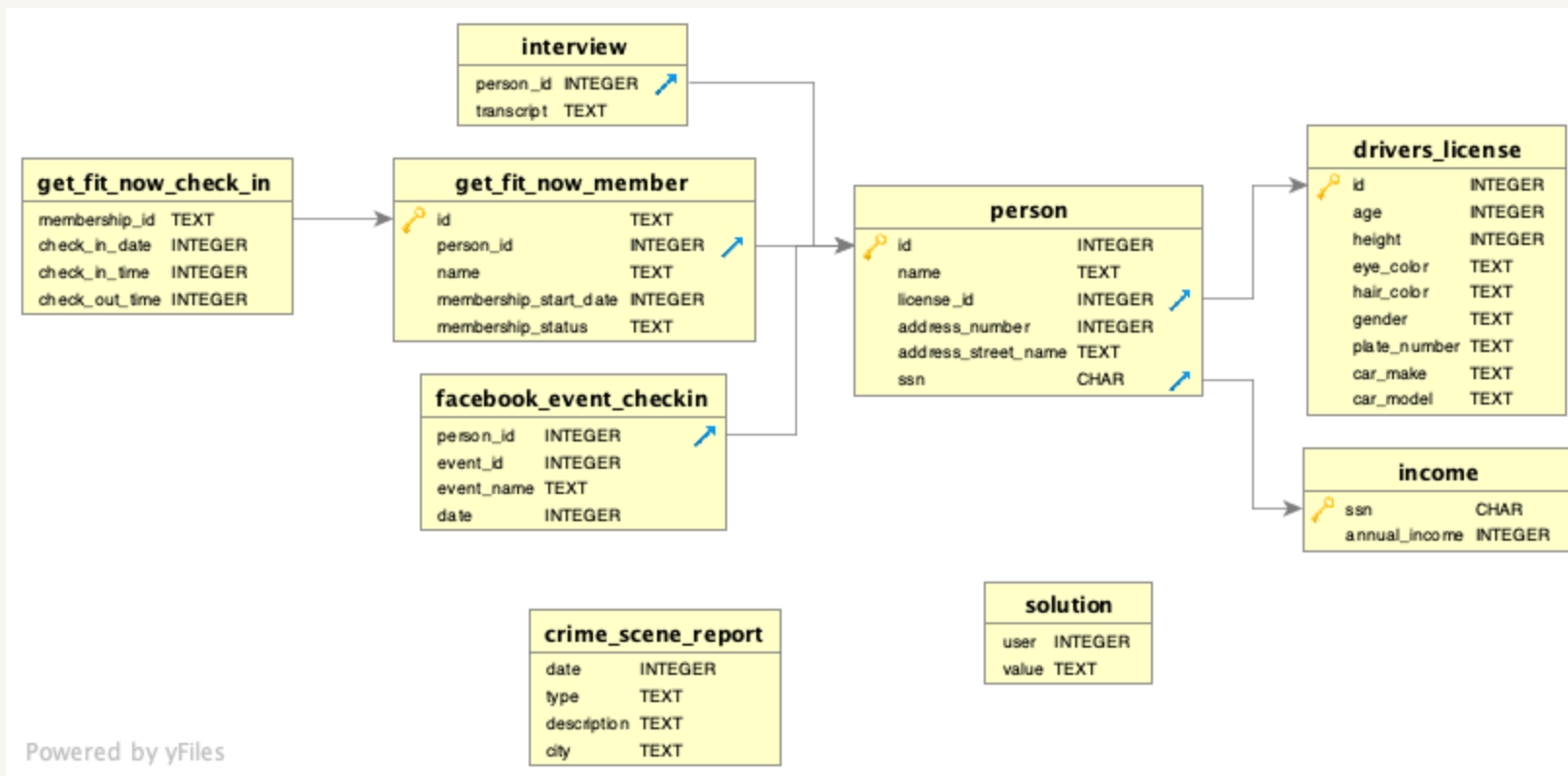
Relationships:

- Customers place Orders
- Orders contain Products via Orderline



◀ Recap

- `get_fit_now_member` ➔ `drivers_license` ?
- `facebook_event_checkin` ➔ `income` ?



The suspect

```
SELECT * FROM person
-- suspect person_id from first witness
WHERE id = 67318
-- suspect person_id from second witness
AND id IN (67318, 28819)
```

The suspect's interview

id	person_id	name	membership_start_date	membership_status
48Z55	67318	Jeremy Bowers	20160101	gold

```
SELECT * FROM interview WHERE person_id = 67318
```

I was hired by a woman with a lot of money. I don't know her name but I know she's around 5'5" (65") or 5'7" (67"). She has red hair and she drives a Tesla Model S. I know that she attended the SQL Symphony Concert 3 times in December 2017.

- **1** red hair, Tesla Model S, height 65" or 67"
- **2** attended SQL Symphony Concert 3 times in December 2017

Filter `drivers_license` by the description

```
SELECT *  
FROM drivers_license  
WHERE hair_color = 'red'  
AND car_make = 'Tesla'  
AND car_model = 'Model S'  
AND height BETWEEN 65 AND 67
```

id	age	height	eye_color	hair_color	gender	plate_number	car_make	car_model
202298	68	66	green	red	female	500123	Tesla	Model S
291182	65	66	blue	red	female	08CM64	Tesla	Model S
918773	48	65	black	red	female	917UU3	Tesla	Model S

Cross-check with the **person** table

id	age	height	eye_color	hair_color	gender	plate_number	car_make	car_model
202298	68	66	green	red	female	500123	Tesla	Model S
291182	65	66	blue	red	female	08CM64	Tesla	Model S
918773	48	65	black	red	female	917UU3	Tesla	Model S

WHERE column **IN** (list of values)

```
SELECT *  
FROM person  
WHERE license_id IN (918773, 291182, 202298)
```

Cross-check using Subquery

Subquery: a query within a query

The output of one query becomes the input of another query



Clue #1 using Subquery

1 Query to get the list of `ID` s from `drivers_license` table that match the description

```
-- It must return the column that we want to use in the outer query
SELECT id
FROM drivers_license
WHERE hair_color = 'red'
AND car_make = 'Tesla'
AND car_model = 'Model S'
AND height BETWEEN 65 AND 67
```

id
202298
291182
918773

Clue #1 using Subquery

2 Insert that query in the `WHERE` clause

`WHERE` column `IN` (subquery returning a list of one column)

```
SELECT *  
FROM person  
-- Instead of hard-coding the list of IDs  
-- where license_id in (918773, 291182, 202298)  
-- Use the subquery that returns that list  
WHERE license_id IN (  
    SELECT id  
    FROM drivers_license  
    WHERE hair_color = 'red'  
    AND car_make = 'Tesla'  
    AND car_model = 'Model S'  
    AND height BETWEEN 65 AND 67  
)
```

Redo the questions with a Subquery

1. Find the interview of the first witness, who lives at the last house on "Northwestern Dr."

- **1** Query to get the person ID of the last house on Northwestern Dr.
- **2** Use the query to get the interview

1 Query to get the person ID of the last house on Northwestern Dr.

```
SELECT id FROM person
WHERE address_street_name = 'Northwestern Dr'
ORDER BY address_number DESC LIMIT 1
```

2 Use the query to get the interview

```
SELECT * FROM interview
WHERE person_id = (
    SELECT id FROM person
    WHERE address_street_name = 'Northwestern Dr'
    ORDER BY address_number DESC LIMIT 1
)
```

!! A subquery with `LIMIT` must use `=`, not `IN` (in MariaDB)



2. Find the first suspect from person table: membership id starts with "48Z" and license plate includes "H42W"

- Subquery 1: membership id starts with "48Z"
- Subquery 2: license plate includes "H42W"
- Outer query: find the person who matches both subqueries



3. Pull up the interview of the second witness. Her name is Annabel and she lives somewhere on "Franklin Ave"

- Subquery 1: find the person ID of Annabel who lives on Franklin Ave
- Outer query: find the interview of that person ID

Three clues in one query

It works, but can you read it?

```
SELECT * FROM person
-- 7. From first witness:
-- gym membership id starts with "48Z" and license plate includes "H42W"
WHERE id IN (SELECT person_id FROM get_fit_now_member WHERE id LIKE '%48Z%')
AND license_id IN (SELECT id FROM drivers_license WHERE plate_number LIKE '%H42W%')
-- 8. From second witness:
-- the suspect was at the gym between 4 PM and 5 PM
AND id IN (
    SELECT person_id FROM get_fit_now_member
    WHERE id IN (
        SELECT membership_id FROM get_fit_now_check_in
        WHERE check_in_time <= 1700
        AND check_out_time >= 1600
    )
)
```

Common Table Expressions (CTE)

Temporary named result set that you can reference within another sql statement

```
WITH cte_name AS (subquery)
```

```
-- Define the CTE
WITH suspect_gym AS (
    SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'
)

-- You can reference the CTE like a regular table
SELECT * FROM person WHERE id IN (SELECT * FROM suspect_gym);
```

Rewrite with CTEs

1 Define multiple CTEs for each clue

WITH cte_name1 AS (subquery1), cte_name2 AS (subquery2), ...

```
WITH
-- membership id starts with "48Z"
suspect_gym AS (SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'),
-- license plate includes "H42W"
suspect_license AS (SELECT id FROM drivers_license WHERE plate_number LIKE '%H42W%'),
-- the suspect was at the gym between 4 PM and 5 PM
suspect_gym_checkin_time AS (
    SELECT person_id FROM get_fit_now_member
    WHERE id IN (
        SELECT membership_id FROM get_fit_now_check_in
        WHERE check_in_time <= 1700
        AND check_out_time >= 1600
    )
)
```

Rewrite with CTEs (cont.)

- 2 Use the CTEs in the main query

```
-- ... WITH block from the previous slide ...  
  
SELECT * FROM person  
WHERE id IN (SELECT * FROM suspect_gym)  
AND license_id IN (SELECT * FROM suspect_license)  
AND id IN (SELECT * FROM suspect_gym_checkin_time)
```

!! CTE can be referenced only once

```
WITH suspect_gym AS (  
    SELECT person_id FROM get_fit_now_member WHERE id LIKE '48Z%'  
)  
  
SELECT * FROM suspect_gym;  
SELECT * FROM suspect_gym; -- ERROR: suspect_gym can be used only once
```

Clue #2

- 1** ~~red hair, Tesla Model S, height 65" or 67"~~
- 2** attended SQL Symphony Concert 3 times in December 2017

Select all attendance of SQL Symphony Concert in 2017

Table: facebook_event_checkin

Columns: all

Rows:

- SQL Symphony Concert
- December 2017

```
SELECT *  
FROM facebook_event_checkin  
WHERE event_name LIKE '%SQL Symphony Concert%'  
AND date BETWEEN 20170101 AND 20171231  
ORDER BY person_id
```

Aggregate functions

- `count()`
- `sum()`
- `avg()`
- `max()`
- `min()`
- `stddev()` : standard deviation
- `variance()` : variance
- ...

<https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html>

`count(*)` to count the number of rows

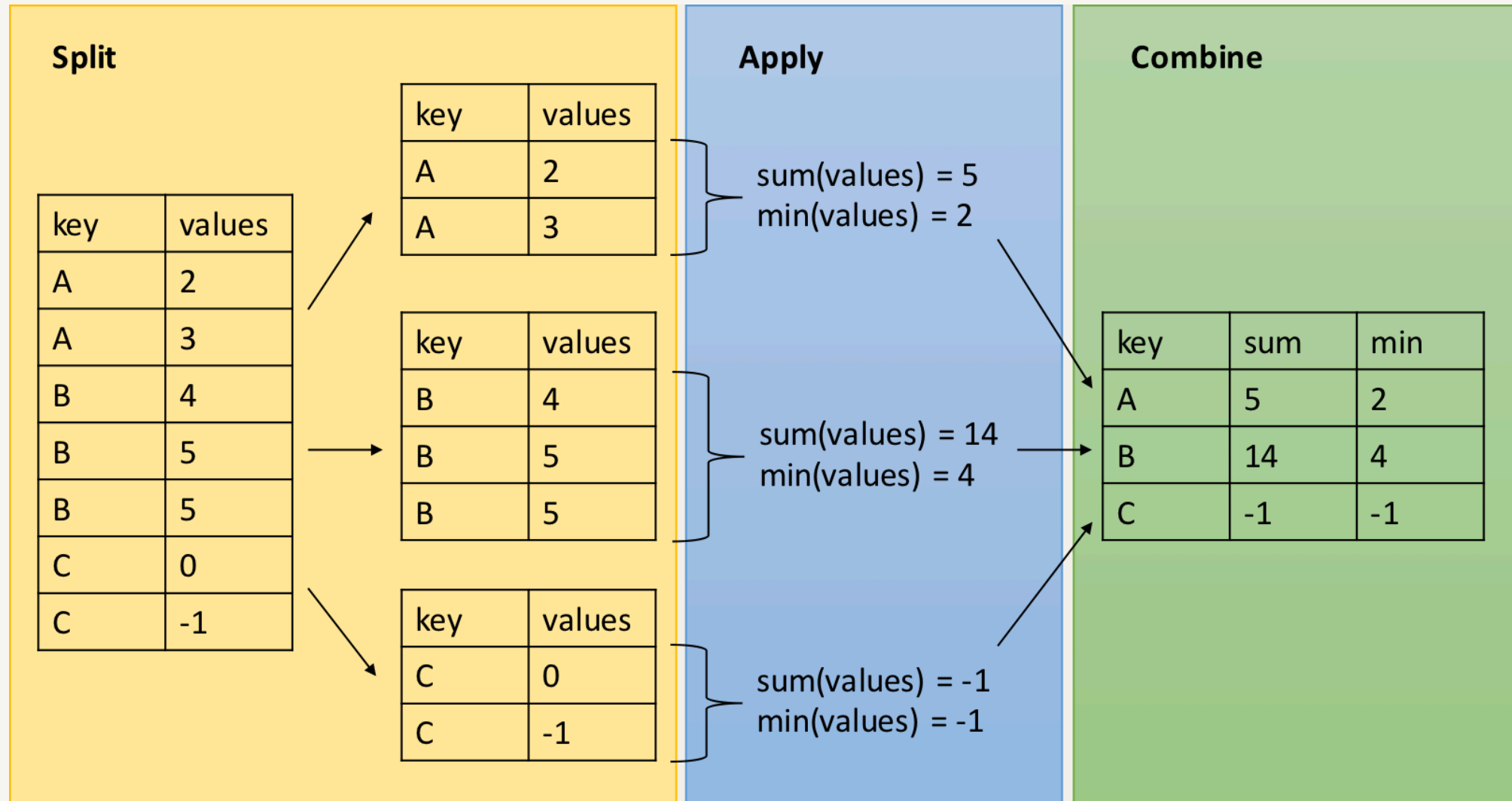
```
SELECT count(*)  
FROM facebook_event_checkin  
WHERE event_name LIKE '%SQL Symphony Concert%'  
AND date BETWEEN 20170101 AND 20171231  
ORDER BY person_id
```

SELECT column **FROM** table **GROUP BY** column

group by :

- group rows that have the same values in specified columns
- "find the number of attendance for each person"

Group rows by columns, apply aggregate functions to each group, and combine the results



GROUP BY person to count each person's attendance

- Group rows by person_id
- Apply count function to each group
- Combine the results

AS alias: name the new column so later clauses can refer to it

```
SELECT person_id, count(*) AS num_attendance
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20170101 AND 20171231
GROUP BY person_id
```

WHERE filters rows, HAVING filters groups

```
SELECT person_id, count(*) AS num_attendance
FROM facebook_event_checkin

-- filter rows before grouping
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20170101 AND 20171231
GROUP BY person_id

-- filter groups after grouping
-- HAVING count(*) = 3 also works
HAVING num_attendance = 3
```

DISTINCT to remove duplicates

Q: On how many dates was the SQL Symphony Concert held in 2017?

➡ Each check-in is one row, so `count(*)` counts people. Count the distinct dates instead.

```
SELECT count(DISTINCT date)
FROM facebook_event_checkin
WHERE event_name LIKE '%SQL Symphony Concert%'
AND date BETWEEN 20170101 AND 20171231
```



From Facebook Event Checkin table

1. What is the earliest check-in date in the table?
2. Which five events had the most check-ins?
3. Who attended three or more different events in December 2017? (several check-ins to the same event count as one)

Putting it All Together

1. Clue #1: People that match the description (`license_id` from `drivers_license`)
2. Clue #2: People that attended SQL Symphony Concert 3 times in 2017 (`person_id` from `facebook_event_checkin`)

Using Subquery

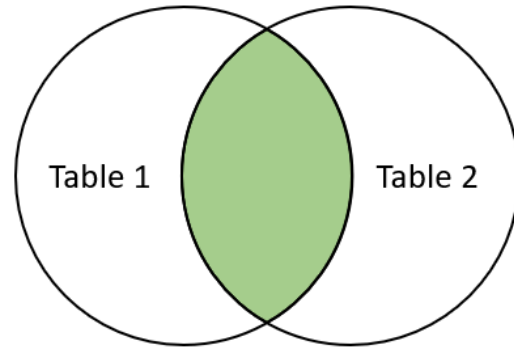
```
SELECT *
FROM person
WHERE license_id IN (
    SELECT id
    FROM drivers_license
    WHERE hair_color = 'red'
    AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
)
AND id IN (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE '%SQL Symphony Concert%'
    AND date BETWEEN 20170101 AND 20171231
    GROUP BY person_id
    HAVING count(*)=3
)
```

Using Subquery and CTE

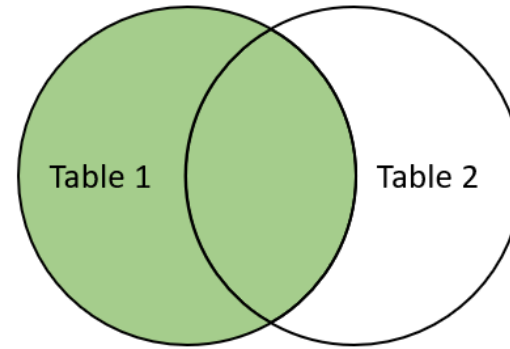
```
WITH
suspect_license AS (
    SELECT id
    FROM drivers_license
    WHERE hair_color = 'red'
    AND car_make = 'Tesla'
    AND car_model = 'Model S'
    AND height BETWEEN 65 AND 67
),
suspect_event AS (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE '%SQL Symphony Concert%'
    AND date BETWEEN 20170101 AND 20171231
    GROUP BY person_id
    HAVING count(*)=3
)

SELECT *
FROM person
WHERE license_id IN (SELECT * FROM suspect_license)
AND id IN (SELECT * FROM suspect_event)
```

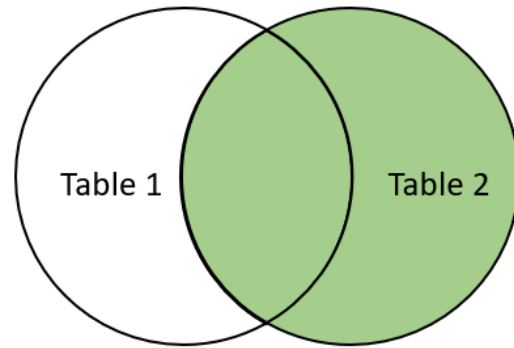
Using **JOIN**



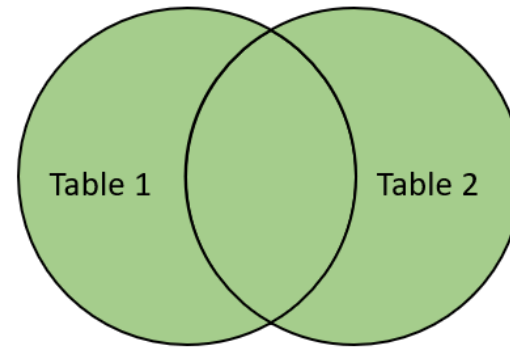
INNER JOIN



LEFT JOIN



RIGHT JOIN



CROSS JOIN

Examples

id	name	license_id
1	Alice	101
2	Bob	102
3	Charlie	103

id	hair_color	car_make
101	red	Tesla
102	blue	Ford
104	black	BMW

JOIN ON

person.license_id = drivers_license.id

- **Inner Join: 101, 102**
- Left Join: 101, 102, 103
- Right Join: 101, 102, 104
- Cross Join: 3 x 3 = 9 rows

JOIN ON (inner join)

- What tables do you want to join?: `person` and `drivers_license`
- What column(s) do you want to join on?: `license_id` from `person` and `id` from `drivers_license`

```
SELECT * FROM table1 JOIN table2 ON table1.column = table2.column
```

```
-- both tables have a column named id, so say which one: table.column
SELECT person.id, drivers_license.id
-- tables to join
FROM person JOIN drivers_license
-- join condition
ON person.license_id = drivers_license.id
```

What column(s) do you want to join on?

1. Join `person` and `drivers_license` on?
2. Join `person` and `facebook_event_checkin` on?

Join `person` and `drivers_license`

```
SELECT *  
FROM person JOIN drivers_license  
ON person.license_id = drivers_license.id  
WHERE hair_color = 'red'  
AND car_make = 'Tesla'  
AND car_model = 'Model S'  
AND height BETWEEN 65 AND 67
```

Putting it All Together with JOIN and Subquery

- JOIN drivers_license : its columns are needed in the result and in WHERE
- Subquery on facebook_event_checkin : only used to decide who qualifies

```
SELECT *
FROM person JOIN drivers_license
ON person.license_id = drivers_license.id
WHERE hair_color = "red" AND car_make = "Tesla"
AND car_model = "Model S"
AND height BETWEEN 65 AND 67
AND person.id IN (
    SELECT person_id
    FROM facebook_event_checkin
    WHERE event_name LIKE "%SQL Symphony Concert%"
    AND date BETWEEN 20170101 AND 20171231
    GROUP BY person_id
    HAVING count(*)=3
)
```



JOIN to show columns from another table

1. Show the second witness's name next to her interview transcript. (Annabel, lives on "Franklin Ave"; tables: `person`, `interview`)
2. The suspect's interview said "a woman with a lot of money". Add each red-haired Tesla driver's annual income to the previous query. (tables: `person`, `drivers_license`, `income`)

Other JOIN syntaxes

JOIN USING or NATURAL JOIN :

when the columns that you want to join on have the same name in both tables

```
-- Join on license_id column
-- this won't work because drivers_license has no license_id column
SELECT * FROM person JOIN drivers_license USING (license_id);
-- Join on all columns with the same name
-- this won't work because id column is not the same in both tables
SELECT * FROM person NATURAL JOIN drivers_license;
```

WHERE :

Joining condition in WHERE clause instead of JOIN clause

```
SELECT * FROM person, drivers_license
WHERE person.license_id = drivers_license.id
```

LEFT JOIN and RIGHT JOIN

LEFT JOIN :

Table on the left: all rows

Table on the right: only matching rows

```
SELECT * FROM person LEFT JOIN get_fit_now_member ON person.id = get_fit_now_member.person_id
```

RIGHT JOIN :

Table on the left: only matching rows

Table on the right: all rows

```
SELECT * FROM person RIGHT JOIN get_fit_now_member ON person.id = get_fit_now_member.person_id
```

Wrap up

- Subquery: query within a query
- Common Table Expressions (CTE): temporary named result set
- Aggregate functions: `count()` , `sum()` , `avg()` , `max()` , `min()` , ...
- `GROUP BY` : group rows by column(s)
- `HAVING` : filter groups after grouping
- `JOIN` : combine data from multiple tables